

开源软件基础

Pandas

Zhilei Ren



OSCAR Team, School of Software, Dalian University of Technology

December 29, 2017



Outline

1 10 Minutes to pandas

Outline

1 10 Minutes to pandas

10 Minutes to pandas

This is a short introduction to pandas, geared mainly for new users.
You can see more complex recipes in the Cookbook

10 Minutes to pandas

Customarily, we import as follows:

10 Minutes to pandas

```
In [1]: import pandas as pd
```

```
In [2]: import numpy as np
```

```
In [3]: import matplotlib.pyplot as plt
```

10 Minutes to pandas

See the Data Structure Intro section

10 Minutes to pandas

Creating a Series by passing a list of values, letting pandas create a default integer index:

10 Minutes to pandas

```
In [4]: s = pd.Series([1,3,5,np.nan,6,8])
```

```
In [5]: s
```

```
Out[5]:
```

```
0      1.0
```

```
1      3.0
```

```
2      5.0
```

```
3      NaN
```

```
4      6.0
```

```
5      8.0
```

```
dtype: float64
```

10 Minutes to pandas

Creating a DataFrame by passing a numpy array, with a datetime index and labeled columns:

10 Minutes to pandas

```
In [6]: dates = pd.date_range('20130101', periods=6)
```

```
In [7]: dates
```

```
Out [7]:
```

```
DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-03', '2013-01-04',  
              '2013-01-05', '2013-01-06'],  
              dtype='datetime64[ns]', freq='D')
```

```
In [8]: df = pd.DataFrame(np.random.randn(6,4),  
                          index=dates,  
                          columns=list('ABCD'))
```

```
In [9]: df
```

```
Out [9]:
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-05	-0.424972	0.567020	0.276232	-1.087401
2013-01-06	-0.673690	0.113648	-1.478427	0.524988

10 Minutes to pandas

Creating a DataFrame by passing a dict of objects that can be converted to series-like.

10 Minutes to pandas

```
In [10]: df2 = pd.DataFrame({ 'A' : 1.,
.....:                        'B' : pd.Timestamp('20130102'),
.....:                        'C' : pd.Series(1,index=list(range(4)),dtype=
.....:                        'D' : np.array([3] * 4,dtype='int32'),
.....:                        'E' : pd.Categorical(["test","train","test",
.....:                        'F' : 'foo' })
.....:
```

```
In [11]: df2
```

```
Out[11]:
```

	A	B	C	D	E	F
0	1.0	2013-01-02	1.0	3	test	foo
1	1.0	2013-01-02	1.0	3	train	foo
2	1.0	2013-01-02	1.0	3	test	foo
3	1.0	2013-01-02	1.0	3	train	foo

10 Minutes to pandas

Having specific dtypes

10 Minutes to pandas

```
In [12]: df2.dtypes
```

```
Out[12]:
```

```
A          float64
```

```
B    datetime64[ns]
```

```
C          float32
```

```
D          int32
```

```
E          category
```

```
F          object
```

```
dtype: object
```

10 Minutes to pandas

If you're using IPython, tab completion for column names (as well as public attributes) is automatically enabled. Here's a subset of the attributes that will be completed:

10 Minutes to pandas

```
In [13]: df2.<TAB>
df2.A          df2.bool
df2.abs        df2.boxplot
df2.add        df2.C
df2.add_prefix df2.clip
df2.add_suffix df2.clip_lower
df2.align      df2.clip_upper
df2.all        df2.columns
df2.any        df2.combine
df2.append     df2.combine_first
df2.apply      df2.compound
df2.applymap   df2 consolidate
df2.D
```

10 Minutes to pandas

As you can see, the columns A, B, C, and D are automatically tab completed. E is there as well; the rest of the attributes have been truncated for brevity.

10 Minutes to pandas

See the Basics section

10 Minutes to pandas

See the top & bottom rows of the frame

10 Minutes to pandas

```
In [14]: df.head()
```

```
Out [14]:
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-05	-0.424972	0.567020	0.276232	-1.087401

```
In [15]: df.tail(3)
```

```
Out [15]:
```

	A	B	C	D
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-05	-0.424972	0.567020	0.276232	-1.087401
2013-01-06	-0.673690	0.113648	-1.478427	0.524988

10 Minutes to pandas

- 使用 index 获得行标号
- 使用 columns 获得列名
- 使用 values 获得内部数据结构
- 使用 describe 获得统计信息
- 使用 T 转置
- 使用 sort_index 按行标号排序（注：非 inplace）
- 使用 sort_values(列名) 按数值排序（同样非 inplace）

10 Minutes to pandas

```
In [16]: df.index
```

```
Out[16]:
```

```
DatetimeIndex(['2013-01-01', '2013-01-02', '2013-01-03', '2013-01-04',  
              '2013-01-05', '2013-01-06'],  
              dtype='datetime64[ns]', freq='D')
```

```
In [17]: df.columns
```

```
Out[17]: Index(['A', 'B', 'C', 'D'], dtype='object')
```

```
In [18]: df.values
```

```
Out[18]:
```

```
array([[ 0.4691, -0.2829, -1.5091, -1.1356],  
       [ 1.2121, -0.1732,  0.1192, -1.0442],  
       [-0.8618, -2.1046, -0.4949,  1.0718],  
       [ 0.7216, -0.7068, -1.0396,  0.2719],  
       [-0.425 ,  0.567 ,  0.2762, -1.0874],  
       [-0.6737,  0.1136, -1.4784,  0.525 ]])
```

10 Minutes to pandas

```
In [19]: df.describe()
```

```
Out[19]:
```

	A	B	C	D
count	6.000000	6.000000	6.000000	6.000000
mean	0.073711	-0.431125	-0.687758	-0.233103
std	0.843157	0.922818	0.779887	0.973118
min	-0.861849	-2.104569	-1.509059	-1.135632
25%	-0.611510	-0.600794	-1.368714	-1.076610
50%	0.022070	-0.228039	-0.767252	-0.386188
75%	0.658444	0.041933	-0.034326	0.461706
max	1.212112	0.567020	0.276232	1.071804

10 Minutes to pandas

In [20]: df.T

Out [20]:

	2013-01-01	2013-01-02	2013-01-03	2013-01-04	2013-01-05	2013-01-06
A	0.469112	1.212112	-0.861849	0.721555	-0.424972	-0.673690
B	-0.282863	-0.173215	-2.104569	-0.706771	0.567020	0.113648
C	-1.509059	0.119209	-0.494929	-1.039575	0.276232	-1.478427
D	-1.135632	-1.044236	1.071804	0.271860	-1.087401	0.524988

10 Minutes to pandas

```
In [21]: df.sort_index(axis=1, ascending=False)
```

```
Out [21]:
```

	D	C	B	A
2013-01-01	-1.135632	-1.509059	-0.282863	0.469112
2013-01-02	-1.044236	0.119209	-0.173215	1.212112
2013-01-03	1.071804	-0.494929	-2.104569	-0.861849
2013-01-04	0.271860	-1.039575	-0.706771	0.721555
2013-01-05	-1.087401	0.276232	0.567020	-0.424972
2013-01-06	0.524988	-1.478427	0.113648	-0.673690

10 Minutes to pandas

```
In [22]: df.sort_values(by='B')
```

```
Out [22]:
```

	A	B	C	D
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-06	-0.673690	0.113648	-1.478427	0.524988
2013-01-05	-0.424972	0.567020	0.276232	-1.087401

10 Minutes to pandas

- 使用. 列名返回一个序列
- 使用 [x:y] 对行切片
- 使用.loc[x:y, ['a', 'b']] 获取分块
- 使用.loc[x, ['a', 'b']] 获取序列（单列）
- 使用.loc[x, y] 获取单个值
- 使用.at[x, y] 同上
- 使用.iloc 与 loc 类似，除了是按下标
- 使用.iat 与 at 类似，除了是按下标

10 Minutes to pandas

```
In [23]: df['A']  
Out[23]:  
2013-01-01    0.469112  
2013-01-02    1.212112  
2013-01-03   -0.861849  
2013-01-04    0.721555  
2013-01-05   -0.424972  
2013-01-06   -0.673690  
Freq: D, Name: A, dtype: float64
```

10 Minutes to pandas

```
In [24]: df[0:3]
```

```
Out [24]:
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804

```
In [25]: df['20130102':'20130104']
```

```
Out [25]:
```

	A	B	C	D
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860

10 Minutes to pandas

Selecting on a multi-axis by label

10 Minutes to pandas

```
In [27]: df.loc[:, ['A', 'B']]
```

```
Out [27]:
```

	A	B
2013-01-01	0.469112	-0.282863
2013-01-02	1.212112	-0.173215
2013-01-03	-0.861849	-2.104569
2013-01-04	0.721555	-0.706771
2013-01-05	-0.424972	0.567020
2013-01-06	-0.673690	0.113648

10 Minutes to pandas

Showing label slicing, both endpoints are included

10 Minutes to pandas

```
In [28]: df.loc['20130102':'20130104',['A','B']]
```

```
Out[28]:
```

	A	B
2013-01-02	1.212112	-0.173215
2013-01-03	-0.861849	-2.104569
2013-01-04	0.721555	-0.706771

10 Minutes to pandas

Reduction in the dimensions of the returned object

10 Minutes to pandas

```
In [29]: df.loc['20130102',['A','B']]
Out[29]:
A      1.212112
B     -0.173215
Name: 2013-01-02 00:00:00, dtype: float64
```

10 Minutes to pandas

For getting a scalar value

10 Minutes to pandas

```
In [30]: df.loc[dates[0], 'A']  
Out[30]: 0.46911229990718628
```

10 Minutes to pandas

For getting fast access to a scalar (equiv to the prior method)

10 Minutes to pandas

```
In [31]: df.at[dates[0], 'A']  
Out[31]: 0.46911229990718628
```


10 Minutes to pandas

See more in Selection by Position

10 Minutes to pandas

Select via the position of the passed integers

10 Minutes to pandas

```
In [32]: df.iloc[3]
```

```
Out[32]:
```

```
A    0.721555
```

```
B   -0.706771
```

```
C   -1.039575
```

```
D    0.271860
```

```
Name: 2013-01-04 00:00:00, dtype: float64
```

10 Minutes to pandas

By integer slices, acting similar to numpy/python

10 Minutes to pandas

```
In [33]: df.iloc[3:5,0:2]
```

```
Out [33]:
```

	A	B
2013-01-04	0.721555	-0.706771
2013-01-05	-0.424972	0.567020

10 Minutes to pandas

By lists of integer position locations, similar to the numpy/python style

10 Minutes to pandas

```
In [34]: df.iloc[[1,2,4],[0,2]]
```

```
Out [34]:
```

	A	C
2013-01-02	1.212112	0.119209
2013-01-03	-0.861849	-0.494929
2013-01-05	-0.424972	0.276232

10 Minutes to pandas

For slicing rows explicitly

10 Minutes to pandas

```
In [35]: df.iloc[1:3,:]
```

```
Out [35]:
```

	A	B	C	D
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804

10 Minutes to pandas

For slicing columns explicitly

10 Minutes to pandas

```
In [36]: df.iloc[:,1:3]
```

```
Out [36]:
```

	B	C
2013-01-01	-0.282863	-1.509059
2013-01-02	-0.173215	0.119209
2013-01-03	-2.104569	-0.494929
2013-01-04	-0.706771	-1.039575
2013-01-05	0.567020	0.276232
2013-01-06	0.113648	-1.478427

10 Minutes to pandas

For getting a value explicitly

10 Minutes to pandas

```
In [37]: df.iloc[1,1]  
Out[37]: -0.17321464905330858
```

10 Minutes to pandas

For getting fast access to a scalar (equiv to the prior method)

10 Minutes to pandas

```
In [38]: df.iat[1,1]  
Out[38]: -0.17321464905330858
```

10 Minutes to pandas

使用 `df[df[列名] 条件]` 筛选列条件除了 `><=!=` 可以用 `isin` 表示包含

10 Minutes to pandas

```
In [39]: df[df.A > 0]
```

```
Out [39]:
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-04	0.721555	-0.706771	-1.039575	0.271860

10 Minutes to pandas

Selecting values from a DataFrame where a boolean condition is met.

10 Minutes to pandas

```
In [40]: df[df > 0]
```

```
Out [40]:
```

	A	B	C	D
2013-01-01	0.469112	NaN	NaN	NaN
2013-01-02	1.212112	NaN	0.119209	NaN
2013-01-03	NaN	NaN	NaN	1.071804
2013-01-04	0.721555	NaN	NaN	0.271860
2013-01-05	NaN	0.567020	0.276232	NaN
2013-01-06	NaN	0.113648	NaN	0.524988

10 Minutes to pandas

Using the `isin()` method for filtering:

10 Minutes to pandas

```
In [41]: df2 = df.copy()
```

```
In [42]: df2['E'] = ['one', 'one', 'two', 'three', 'four', 'three']
```

```
In [43]: df2
```

```
Out[43]:
```

	A	B	C	D	E
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632	one
2013-01-02	1.212112	-0.173215	0.119209	-1.044236	one
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804	two
2013-01-04	0.721555	-0.706771	-1.039575	0.271860	three
2013-01-05	-0.424972	0.567020	0.276232	-1.087401	four
2013-01-06	-0.673690	0.113648	-1.478427	0.524988	three

```
In [44]: df2[df2['E'].isin(['two', 'four'])]
```

```
Out[44]:
```

	A	B	C	D	E
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804	two
2013-01-05	-0.424972	0.567020	0.276232	-1.087401	four

10 Minutes to pandas

Setting a new column automatically aligns the data by the indexes

10 Minutes to pandas

```
In [45]: s1 = pd.Series([1,2,3,4,5,6], index=pd.date_range('20130102', per
```

```
In [46]: s1
```

```
Out[46]:
```

```
2013-01-02    1
```

```
2013-01-03    2
```

```
2013-01-04    3
```

```
2013-01-05    4
```

```
2013-01-06    5
```

```
2013-01-07    6
```

```
Freq: D, dtype: int64
```

```
In [47]: df['F'] = s1
```

10 Minutes to pandas

Setting values by label

10 Minutes to pandas

```
In [48]: df.at[dates[0], 'A'] = 0
```

10 Minutes to pandas

Setting values by position

10 Minutes to pandas

```
In [49]: df.iat[0,1] = 0
```

10 Minutes to pandas

Setting by assigning with a numpy array

10 Minutes to pandas

```
In [50]: df.loc[:, 'D'] = np.array([5] * len(df))
```

10 Minutes to pandas

The result of the prior setting operations

10 Minutes to pandas

In [51]: df

Out [51]:

	A	B	C	D	F
2013-01-01	0.000000	0.000000	-1.509059	5	NaN
2013-01-02	1.212112	-0.173215	0.119209	5	1.0
2013-01-03	-0.861849	-2.104569	-0.494929	5	2.0
2013-01-04	0.721555	-0.706771	-1.039575	5	3.0
2013-01-05	-0.424972	0.567020	0.276232	5	4.0
2013-01-06	-0.673690	0.113648	-1.478427	5	5.0

10 Minutes to pandas

A where operation with setting.

10 Minutes to pandas

```
In [52]: df2 = df.copy()
```

```
In [53]: df2[df2 > 0] = -df2
```

```
In [54]: df2
```

```
Out [54]:
```

	A	B	C	D	F
2013-01-01	0.000000	0.000000	-1.509059	-5	NaN
2013-01-02	-1.212112	-0.173215	-0.119209	-5	-1.0
2013-01-03	-0.861849	-2.104569	-0.494929	-5	-2.0
2013-01-04	-0.721555	-0.706771	-1.039575	-5	-3.0
2013-01-05	-0.424972	-0.567020	-0.276232	-5	-4.0
2013-01-06	-0.673690	-0.113648	-1.478427	-5	-5.0

10 Minutes to pandas

pandas primarily uses the value `np.nan` to represent missing data. It is by default not included in computations. See the Missing Data section

10 Minutes to pandas

Reindexing allows you to change/add/delete the index on a specified axis. This returns a copy of the data. 使用 `reindex` 修改行/列，但若有未出现数据，对应行/列会设置为 `NaN` 使用 `dropna(how='any')` 删除 `NaN` 列，`'any'` 表示任意，`'all'` 表示所有使用 `fillna` 替换 `NaN` 值

10 Minutes to pandas

```
In [55]: df1 = df.reindex(index=dates[0:4], columns=list(df.columns) + ['E'])
```

```
In [56]: df1.loc[dates[0]:dates[1], 'E'] = 1
```

```
In [57]: df1
```

```
Out[57]:
```

	A	B	C	D	F	E
2013-01-01	0.000000	0.000000	-1.509059	5	NaN	1.0
2013-01-02	1.212112	-0.173215	0.119209	5	1.0	1.0
2013-01-03	-0.861849	-2.104569	-0.494929	5	2.0	NaN
2013-01-04	0.721555	-0.706771	-1.039575	5	3.0	NaN

10 Minutes to pandas

To drop any rows that have missing data.

10 Minutes to pandas

```
In [58]: df1.dropna(how='any')
```

```
Out [58]:
```

	A	B	C	D	F	E
2013-01-02	1.212112	-0.173215	0.119209	5	1.0	1.0

10 Minutes to pandas

Filling missing data

10 Minutes to pandas

```
In [59]: df1.fillna(value=5)
```

```
Out [59]:
```

	A	B	C	D	F	E
2013-01-01	0.000000	0.000000	-1.509059	5	5.0	1.0
2013-01-02	1.212112	-0.173215	0.119209	5	1.0	1.0
2013-01-03	-0.861849	-2.104569	-0.494929	5	2.0	5.0
2013-01-04	0.721555	-0.706771	-1.039575	5	3.0	5.0

10 Minutes to pandas

To get the boolean mask where values are nan

10 Minutes to pandas

```
In [60]: pd.isna(df1)
```

```
Out[60]:
```

	A	B	C	D	F	E
2013-01-01	False	False	False	False	True	False
2013-01-02	False	False	False	False	False	False
2013-01-03	False	False	False	False	False	True
2013-01-04	False	False	False	False	False	True

10 Minutes to pandas

See the Basic section on Binary Ops

10 Minutes to pandas

Operations in general exclude missing data.

10 Minutes to pandas

Performing a descriptive statistic mean 表示均值, 0 按列, 1 按行对某一列, 使用 value_counts 获取频率 pandas.concat([df1, df2]) 拼接 DataFrame 两个 DataFrame 用 merge 归并, 类似 SQL 的连接使用 groupby 进行分组

10 Minutes to pandas

```
In [61]: df.mean()  
Out[61]:  
A    -0.004474  
B    -0.383981  
C    -0.687758  
D     5.000000  
F     3.000000  
dtype: float64
```

10 Minutes to pandas

```
In [62]: df.mean(1)
Out[62]:
2013-01-01    0.872735
2013-01-02    1.431621
2013-01-03    0.707731
2013-01-04    1.395042
2013-01-05    1.883656
2013-01-06    1.592306
Freq: D, dtype: float64
```

10 Minutes to pandas

```
In [68]: s = pd.Series(np.random.randint(0, 7, size=10))
```

```
In [69]: s
```

```
Out[69]:
```

```
0      4
```

```
1      2
```

```
2      1
```

```
3      2
```

```
4      6
```

```
5      4
```

```
6      4
```

```
7      6
```

```
8      4
```

```
9      4
```

```
dtype: int64
```

```
In [70]: s.value_counts()
```

```
Out[70]:
```

```
4      5
```

```
6      2
```

```
2      2
```


10 Minutes to pandas

Series is equipped with a set of string processing methods in the `str` attribute that make it easy to operate on each element of the array, as in the code snippet below. Note that pattern-matching in `str` generally uses regular expressions by default (and in some cases always uses them). See more at [Vectorized String Methods](#).

10 Minutes to pandas

```
In [71]: s = pd.Series(['A', 'B', 'C', 'Aaba', 'Baca', np.nan, 'CABA', 'dog', 'cat'])
```

```
In [72]: s.str.lower()
```

```
Out[72]:
```

```
0      a
```

```
1      b
```

```
2      c
```

```
3    aaba
```

```
4    baca
```

```
5     NaN
```

```
6    caba
```

```
7     dog
```

```
8     cat
```

```
dtype: object
```

10 Minutes to pandas

pandas provides various facilities for easily combining together Series, DataFrame, and Panel objects with various kinds of set logic for the indexes and relational algebra functionality in the case of join / merge-type operations.

10 Minutes to pandas

See the Merging section

10 Minutes to pandas

Concatenating pandas objects together with `concat()`:

10 Minutes to pandas

```
In [73]: df = pd.DataFrame(np.random.randn(10, 4))
```

```
In [74]: df
```

```
Out[74]:
```

	0	1	2	3
0	-0.548702	1.467327	-1.015962	-0.483075
1	1.637550	-1.217659	-0.291519	-1.745505
2	-0.263952	0.991460	-0.919069	0.266046
3	-0.709661	1.669052	1.037882	-1.705775
4	-0.919854	-0.042379	1.247642	-0.009920
5	0.290213	0.495767	0.362949	1.548106
6	-1.131345	-0.089329	0.337863	-0.945867
7	-0.932132	1.956030	0.017587	-0.016692
8	-0.575247	0.254161	-1.143704	0.215897
9	1.193555	-0.077118	-0.408530	-0.862495

```
# break it into pieces
```

```
In [75]: pieces = [df[:3], df[3:7], df[7:]]
```

```
In [76]: pd.concat(pieces)
```

```
Out[76]:
```

10 Minutes to pandas

SQL style merges. See the Database style joining

10 Minutes to pandas

```
In [77]: left = pd.DataFrame({'key': ['foo', 'foo'], 'lval': [1, 2]})
```

```
In [78]: right = pd.DataFrame({'key': ['foo', 'foo'], 'rval': [4, 5]})
```

```
In [79]: left
```

```
Out[79]:
```

	key	lval
0	foo	1
1	foo	2

```
In [80]: right
```

```
Out[80]:
```

	key	rval
0	foo	4
1	foo	5

```
In [81]: pd.merge(left, right, on='key')
```

```
Out[81]:
```

	key	lval	rval
0	foo	1	4
1	foo	1	5

10 Minutes to pandas

Another example that can be given is:

10 Minutes to pandas

```
In [82]: left = pd.DataFrame({'key': ['foo', 'bar'], 'lval': [1, 2]})
```

```
In [83]: right = pd.DataFrame({'key': ['foo', 'bar'], 'rval': [4, 5]})
```

```
In [84]: left
```

```
Out[84]:
```

	key	lval
0	foo	1
1	bar	2

```
In [85]: right
```

```
Out[85]:
```

	key	rval
0	foo	4
1	bar	5

```
In [86]: pd.merge(left, right, on='key')
```

```
Out[86]:
```

	key	lval	rval
0	foo	1	4
1	bar	2	5

10 Minutes to pandas

Append rows to a dataframe. See the Appending

10 Minutes to pandas

```
In [87]: df = pd.DataFrame(np.random.randn(8, 4), columns=['A', 'B', 'C', 'D'])
```

```
In [88]: df
```

```
Out[88]:
```

	A	B	C	D
0	1.346061	1.511763	1.627081	-0.990582
1	-0.441652	1.211526	0.268520	0.024580
2	-1.577585	0.396823	-0.105381	-0.532532
3	1.453749	1.208843	-0.080952	-0.264610
4	-0.727965	-0.589346	0.339969	-0.693205
5	-0.339355	0.593616	0.884345	1.591431
6	0.141809	0.220390	0.435589	0.192451
7	-0.096701	0.803351	1.715071	-0.708758

```
In [89]: s = df.iloc[3]
```

```
In [90]: df.append(s, ignore_index=True)
```

```
Out[90]:
```

	A	B	C	D
0	1.346061	1.511763	1.627081	-0.990582
1	-0.441652	1.211526	0.268520	0.024580

10 Minutes to pandas

By “group by” we are referring to a process involving one or more of the following steps

10 Minutes to pandas

```
In [91]: df = pd.DataFrame({'A' : ['foo', 'bar', 'foo', 'bar',
.....:                             'foo', 'bar', 'foo', 'foo'],
.....:                     'B' : ['one', 'one', 'two', 'three',
.....:                             'two', 'two', 'one', 'three'],
.....:                     'C' : np.random.randn(8),
.....:                     'D' : np.random.randn(8)})
.....:
```

```
In [92]: df
```

```
Out[92]:
```

	A	B	C	D
0	foo	one	-1.202872	-0.055224
1	bar	one	-1.814470	2.395985
2	foo	two	1.018601	1.552825
3	bar	three	-0.595447	0.166599
4	foo	two	1.395433	0.047609
5	bar	two	-0.392670	-0.136473
6	foo	one	0.007207	-0.561757
7	foo	three	1.928123	-1.623033

10 Minutes to pandas

Grouping and then applying a function sum to the resulting groups.

10 Minutes to pandas

```
In [93]: df.groupby('A').sum()
```

```
Out[93]:
```

	C	D
A		
bar	-2.802588	2.42611
foo	3.146492	-0.63958

10 Minutes to pandas

```
In [94]: df.groupby(['A', 'B']).sum()
```

```
Out[94]:
```

		C	D
A	B		
bar	one	-1.814470	2.395985
	three	-0.595447	0.166599
	two	-0.392670	-0.136473
foo	one	-1.195665	-0.616981
	three	1.928123	-1.623033
	two	2.414034	1.600434

10 Minutes to pandas

pandas has simple, powerful, and efficient functionality for performing resampling operations during frequency conversion (e.g., converting secondly data into 5-minutely data). This is extremely common in, but not limited to, financial applications. See the Time Series section

10 Minutes to pandas

```
In [108]: rng = pd.date_range('1/1/2012', periods=100, freq='S')  
  
In [109]: ts = pd.Series(np.random.randint(0, 500, len(rng)), index=rng)  
  
In [110]: ts.resample('5Min').sum()  
Out[110]:  
2012-01-01      25083  
Freq: 5T, dtype: int64
```

10 Minutes to pandas

Time zone representation

10 Minutes to pandas

```
In [111]: rng = pd.date_range('3/6/2012 00:00', periods=5, freq='D')
```

```
In [112]: ts = pd.Series(np.random.randn(len(rng)), rng)
```

```
In [113]: ts
```

```
Out[113]:
```

```
2012-03-06    0.464000
2012-03-07    0.227371
2012-03-08   -0.496922
2012-03-09    0.306389
2012-03-10   -2.290613
Freq: D, dtype: float64
```

```
In [114]: ts_utc = ts.tz_localize('UTC')
```

```
In [115]: ts_utc
```

```
Out[115]:
```

```
2012-03-06 00:00:00+00:00    0.464000
2012-03-07 00:00:00+00:00    0.227371
2012-03-08 00:00:00+00:00   -0.496922
2012-03-09 00:00:00+00:00    0.306389
```

10 Minutes to pandas

Convert to another time zone

10 Minutes to pandas

```
In [116]: ts_utc.tz_convert('US/Eastern')
Out[116]:
2012-03-05 19:00:00-05:00    0.464000
2012-03-06 19:00:00-05:00    0.227371
2012-03-07 19:00:00-05:00   -0.496922
2012-03-08 19:00:00-05:00    0.306389
2012-03-09 19:00:00-05:00   -2.290613
Freq: D, dtype: float64
```

10 Minutes to pandas

Converting between time span representations

10 Minutes to pandas

```
In [117]: rng = pd.date_range('1/1/2012', periods=5, freq='M')
```

```
In [118]: ts = pd.Series(np.random.randn(len(rng)), index=rng)
```

```
In [119]: ts
```

```
Out[119]:
```

```
2012-01-31    -1.134623
2012-02-29    -1.561819
2012-03-31    -0.260838
2012-04-30     0.281957
2012-05-31     1.523962
Freq: M, dtype: float64
```

```
In [120]: ps = ts.to_period()
```

```
In [121]: ps
```

```
Out[121]:
```

```
2012-01    -1.134623
2012-02    -1.561819
2012-03    -0.260838
2012-04     0.281957
```

10 Minutes to pandas

Converting between period and timestamp enables some convenient arithmetic functions to be used. In the following example, we convert a quarterly frequency with year ending in November to 9am of the end of the month following the quarter end:

10 Minutes to pandas

```
In [123]: prng = pd.period_range('1990Q1', '2000Q4', freq='Q-NOV')

In [124]: ts = pd.Series(np.random.randn(len(prng)), prng)

In [125]: ts.index = (prng.asfreq('M', 'e') + 1).asfreq('H', 's') + 9

In [126]: ts.head()
Out[126]:
1990-03-01 09:00    -0.902937
1990-06-01 09:00     0.068159
1990-09-01 09:00    -0.057873
1990-12-01 09:00    -0.368204
1991-03-01 09:00    -1.144073
Freq: H, dtype: float64
```

10 Minutes to pandas

pandas can include categorical data in a DataFrame. For full docs, see the categorical introduction and the API documentation.

10 Minutes to pandas

```
In [127]: df = pd.DataFrame({"id": [1, 2, 3, 4, 5, 6], "raw_grade": ['a', 'b', 'b', 'a', 'b', 'a']})
```

10 Minutes to pandas

Convert the raw grades to a categorical data type.

10 Minutes to pandas

```
In [128]: df["grade"] = df["raw_grade"].astype("category")
```

```
In [129]: df["grade"]
```

```
Out[129]:
```

```
0      a
```

```
1      b
```

```
2      b
```

```
3      a
```

```
4      a
```

```
5      e
```

```
Name: grade, dtype: category
```

```
Categories (3, object): [a, b, e]
```

10 Minutes to pandas

Rename the categories to more meaningful names (assigning to `Series.cat.categories` is inplace!)

10 Minutes to pandas

```
In [130]: df["grade"].cat.categories = ["very good", "good", "very bad"]
```

10 Minutes to pandas

Reorder the categories and simultaneously add the missing categories (methods under Series .cat return a new Series per default).

10 Minutes to pandas

```
In [131]: df["grade"] = df["grade"].cat.set_categories(["very bad", "bad",
```

```
In [132]: df["grade"]
```

```
Out[132]:
```

```
0    very good
```

```
1         good
```

```
2         good
```

```
3    very good
```

```
4    very good
```

```
5    very bad
```

```
Name: grade, dtype: category
```

```
Categories (5, object): [very bad, bad, medium, good, very good]
```

10 Minutes to pandas

Sorting is per order in the categories, not lexical order.

10 Minutes to pandas

```
In [133]: df.sort_values(by="grade")
```

```
Out[133]:
```

	id	raw_grade	grade
5	6	e	very bad
1	2	b	good
2	3	b	good
0	1	a	very good
3	4	a	very good
4	5	a	very good

10 Minutes to pandas

Grouping by a categorical column shows also empty categories.

10 Minutes to pandas

```
In [134]: df.groupby("grade").size()
Out[134]:
grade
very bad    1
bad         0
medium      0
good        2
very good   3
dtype: int64
```

10 Minutes to pandas

Plotting docs.

10 Minutes to pandas

```
In [135]: ts = pd.Series(np.random.randn(1000), index=pd.date_range('1/1/2012', periods=1000))
```

```
In [136]: ts = ts.cumsum()
```

```
In [137]: ts.plot()
```

```
Out[137]: <matplotlib.axes._subplots.AxesSubplot at 0x1122ad630>
```

10 Minutes to pandas

On DataFrame, `plot()` is a convenience to plot all of the columns with labels:

10 Minutes to pandas

```
In [138]: df = pd.DataFrame(np.random.randn(1000, 4), index=ts.index,  
.....:                      columns=['A', 'B', 'C', 'D'])  
.....:
```

```
In [139]: df = df.cumsum()
```

```
In [140]: plt.figure(); df.plot(); plt.legend(loc='best')  
Out[140]: <matplotlib.legend.Legend at 0x115033cf8>
```

10 Minutes to pandas

Writing to a csv file

10 Minutes to pandas

```
In [141]: df.to_csv('foo.csv')
```

10 Minutes to pandas

Reading from a csv file

10 Minutes to pandas

```
In [142]: pd.read_csv('foo.csv')
```

```
Out[142]:
```

	Unnamed: 0	A	B	C	D
0	2000-01-01	0.266457	-0.399641	-0.219582	1.186860
1	2000-01-02	-1.170732	-0.345873	1.653061	-0.282953
2	2000-01-03	-1.734933	0.530468	2.060811	-0.515536
3	2000-01-04	-1.555121	1.452620	0.239859	-1.156896
4	2000-01-05	0.578117	0.511371	0.103552	-2.428202
5	2000-01-06	0.478344	0.449933	-0.741620	-1.962409
6	2000-01-07	1.235339	-0.091757	-1.543861	-1.084753
..	...	...	...	...	...
993	2002-09-20	-10.628548	-9.153563	-7.883146	28.313940
994	2002-09-21	-10.390377	-8.727491	-6.399645	30.914107
995	2002-09-22	-8.985362	-8.485624	-4.669462	31.367740
996	2002-09-23	-9.558560	-8.781216	-4.499815	30.518439
997	2002-09-24	-9.902058	-9.340490	-4.386639	30.105593
998	2002-09-25	-10.216020	-9.480682	-3.933802	29.758560
999	2002-09-26	-11.856774	-10.671012	-3.216025	29.369368

```
[1000 rows x 5 columns]
```

10 Minutes to pandas

Reading and writing to HDFStores

10 Minutes to pandas

Writing to a HDF5 Store

10 Minutes to pandas

```
In [143]: df.to_hdf('foo.h5','df')
```

10 Minutes to pandas

Reading from a HDF5 Store

10 Minutes to pandas

```
In [144]: pd.read_hdf('foo.h5', 'df')
```

```
Out[144]:
```

	A	B	C	D
2000-01-01	0.266457	-0.399641	-0.219582	1.186860
2000-01-02	-1.170732	-0.345873	1.653061	-0.282953
2000-01-03	-1.734933	0.530468	2.060811	-0.515536
2000-01-04	-1.555121	1.452620	0.239859	-1.156896
2000-01-05	0.578117	0.511371	0.103552	-2.428202
2000-01-06	0.478344	0.449933	-0.741620	-1.962409
2000-01-07	1.235339	-0.091757	-1.543861	-1.084753
...	...	...	...	...
2002-09-20	-10.628548	-9.153563	-7.883146	28.313940
2002-09-21	-10.390377	-8.727491	-6.399645	30.914107
2002-09-22	-8.985362	-8.485624	-4.669462	31.367740
2002-09-23	-9.558560	-8.781216	-4.499815	30.518439
2002-09-24	-9.902058	-9.340490	-4.386639	30.105593
2002-09-25	-10.216020	-9.480682	-3.933802	29.758560
2002-09-26	-11.856774	-10.671012	-3.216025	29.369368

```
[1000 rows x 4 columns]
```

10 Minutes to pandas

Reading and writing to MS Excel

10 Minutes to pandas

Writing to an excel file

10 Minutes to pandas

```
In [145]: df.to_excel('foo.xlsx', sheet_name='Sheet1')
```

10 Minutes to pandas

Reading from an excel file

10 Minutes to pandas

```
In [146]: pd.read_excel('foo.xlsx', 'Sheet1', index_col=None, na_values=['
```

```
Out [146]:
```

	A	B	C	D
2000-01-01	0.266457	-0.399641	-0.219582	1.186860
2000-01-02	-1.170732	-0.345873	1.653061	-0.282953
2000-01-03	-1.734933	0.530468	2.060811	-0.515536
2000-01-04	-1.555121	1.452620	0.239859	-1.156896
2000-01-05	0.578117	0.511371	0.103552	-2.428202
2000-01-06	0.478344	0.449933	-0.741620	-1.962409
2000-01-07	1.235339	-0.091757	-1.543861	-1.084753
...	...	...	...	...
2002-09-20	-10.628548	-9.153563	-7.883146	28.313940
2002-09-21	-10.390377	-8.727491	-6.399645	30.914107
2002-09-22	-8.985362	-8.485624	-4.669462	31.367740
2002-09-23	-9.558560	-8.781216	-4.499815	30.518439
2002-09-24	-9.902058	-9.340490	-4.386639	30.105593
2002-09-25	-10.216020	-9.480682	-3.933802	29.758560
2002-09-26	-11.856774	-10.671012	-3.216025	29.369368

```
[1000 rows x 4 columns]
```

10 Minutes to pandas

If you are trying an operation and you see an exception like:

10 Minutes to pandas

```
>>> if pd.Series([False, True, False]):  
    print("I was true")
```

Traceback

```
...
```

```
ValueError: The truth value of an array is ambiguous. Use a.empty, a.any()
```

10 Minutes to pandas

See Comparisons for an explanation and what to do.

10 Minutes to pandas

See Gotchas as well.